AUTOMATION PORTFOLIO — TECHNICAL RECORD

SOPHON FINANCE SYSTEMS

AI-Driven Finance & Accounting Automation | github.com/sophonfinance-wq/finance-automation-portfolio

TRIANGULATE - MULTI-AGENT AI REVIEW

Triangulate is a deterministic, separation-of-duties AI validation framework that runs financial workpapers through multiple independent review roles and a human-gated verdict, so AI can accelerate preparation without becoming an unaccountable single point of failure.

Report No.	SFS-ENG-006	**Package**	triangulate
System	Triangulate - Multi-Agent AI Review	**Data**	Fictional / seeded
Run	`python -m triangulate`	**Status**	Public — portfolio record

1.0 OVERVIEW AND SCENARIO

A preparer submits the fictional FY24-Q4 tax workpaper for Maple Fund LP (engagement ENG-DEMO-0001) in which Total Revenue (cell B5) is stated as 543,000 while its own formula =B2+B3+B4 actually sums to 544,000, the Estimated Tax cell (B7) is hard-coded with no formula on nothing more than an AI-generated tax-rate guess (B6 = 0.18), and an internal process note ("placeholder until Reviewer confirms") leaks into a client-facing field. The framework catches the 1,000 tie-out break independently through both an LLM-style reviewer and a deterministic auditor, ranks the findings, and the human gate returns FAIL rather than signing off. A companion adversarial demo injects a single 49,000 overstatement into an otherwise-clean workpaper and shows it cascade into six Critical tie-out breaks that no plausibility skim would catch.

2.0 WHAT THE SYSTEM AUTOMATES

- Builds a seeded, fully fictional workpaper (Preparer) and drives it through a fixed multi-role validation pipeline end to end with no manual steps and no network or API key by default.
- Runs an adversarial LLM-style reviewer that flags issues strictly by cell reference (tie-out mismatch, hard-coded figures, unsupported AI assumptions, leaked internal/process language) and is structurally prevented from editing the file.
- Re-derives every formula cell mechanically via a deterministic auditor using a safe shunting-yard evaluator (no Python eval), raising Critical tie-out breaks on any re-derivation gap.
- Reconciles and de-duplicates all raised findings by (code, cell reference), resolving conflicts by a six-level source-of-truth authority hierarchy and then a four-level severity taxonomy.
- Applies an explicit human-gate policy that converts the severity profile into a PASS / FLAG / FAIL verdict and assembles a fix packet of the Critical/High items for remediation.
- Emits a reviewer evidence pack to ./output: builder memo, fix packet, change log, QA summary, and a machine-readable verdict.json.
- Offers an optional review loop that auto-re-derives broken formula cells on new hashed workpaper versions, re-runs the full pipeline each turn, and escalates non-arithmetic (judgment/authority) findings to a human.

- Returns a process exit code (0 on PASS, non-zero on FLAG/FAIL) so the pipeline can operate as a CI sign-off gate.

3.0 OPERATING PROCEDURE

1. Build: the Preparer constructs a fresh, seeded fictional workpaper (the only role handed a mutable copy) and writes a builder memo declaring sources, AI assumptions, and risk posture.
2. Review: the adversarial reviewer receives a read-only frozen snapshot and returns findings only; the orchestrator records the workpaper digest before and after.
3. Specialist step: a read-only second opinion (e.g. rate-out-of-band checks) plus a separately, explicitly authorized normalization transform that is logged to the change log.
4. Deterministic audit: the auditor re-derives every formula on a read-only view and checks structural completeness of required summary cells (B5, B7, B8) and tie-outs.
5. Separation-of-duties enforcement: after each read-only step the SHA-256 workpaper digest is compared to the pre-step digest; any change aborts the run with SeparationOfDutiesError.
6. Reconcile: de-duplicate findings by (code, cell reference), letting higher authority then higher severity win, and sort most-severe-first for deterministic output.
7. Human gate: any Critical yields FAIL, any High yields FLAG, otherwise PASS with documented residual Medium/Low notes; the Critical/High subset becomes the fix packet.
8. Emit: write the four audit-trail artifacts plus verdict.json to ./output and return the PASS/non-PASS exit code.

4.0 CONTROLS AND SAFEGUARDS

- Read-only enforcement in code: reviewer, specialist, and auditor receive a ReadOnlyWorkpaperView (backed by a deep copy) whose __setattr__ raises WorkpaperMutationError, so 'flag, never fix' cannot be violated even by accident.
- Mechanical separation of duties: the orchestrator hashes the workpaper (SHA-256 digest) before and after every read-only step and aborts with SeparationOfDutiesError if the digest changes.
- Independent double-detection: tie-out breaks are flagged by both the LLM-style reviewer and the deterministic auditor, so a failed sign-off never rests on a single AI's opinion of its own work.
- Authority hierarchy: a six-level source-of-truth ranking places AI-generated assumptions at the lowest authority; such items cannot be auto-cleared and are escalated to a human for a cited source.
- Deterministic offline default: no API key, no network, and a non-eval formula evaluator mean the same workpaper always yields the same findings; the live Claude reviewer is an import-safe, opt-in stub never invoked by the default pipeline.
- Human gate as an explicit, auditable policy that separates machine detection from final sign-off, recording verdict status, rationale, and signer in verdict.json.

5.0 VERIFICATION AND TESTING

Correctness is pinned by a large offline pytest suite (the README states 1,311 tests; collection in this environment reports 1,320), including parametric formula, model, orchestrator, and reconcile coverage. The load-bearing adversarial claim is guarded by tests/test_adversarial_demo.py, which asserts that the injected figure breaks the tie-out, that cell B5 is flagged independently by both reviewer and auditor, that the error cascades to B5/B7/B8 as Critical, that the CLI exits non-zero on the defective/adversarial run, and that a clean workpaper still exits 0. The committed verdict.json for the defective sample independently shows Critical=2, High=1, Medium=2, Low=1 resolving to FAIL, matching the documented behavior.

6.0 KEY FACTS

Attribute	Value
Automated test suite	1,311 tests (README); 1,320 collected in-environment
Validation roles (separation of duties)	5 - Preparer, Reviewer, Specialist, Deterministic Audit, Human Gate
Authority hierarchy	6 levels (AI assumption -> signed prior-year work)
Severity taxonomy	4 - Critical / High / Medium / Low
Distinct rule/finding codes	9 across reviewer, auditor, and specialist
Adversarial demo injection	$49,000 overstatement cascades into 6 Critical findings -> FAIL

This record documents a self-contained system in the Sophon Finance Systems automation portfolio. All data is fictional and seeded; no employer or client workpaper, entity, methodology, path, or figure is reproduced. Prepared 14 July 2026.